MODULE 4 – EMERGING DATABASE MODELS, TECHNOLOGIES AND APPLICATIONS

Unit 2: Conventional Database Management Systems

1. Introduction:

Oracle has provided high-quality database solutions since the 1970s. The most recent version of Oracle Database was designed to integrate with cloud-based systems, and it allows you to manage massive databases with billions of records. Moreover, Oracle lets you organize data via a "grid framework" and it uses dynamic data masking for an additional layer of security. Traditionally, Oracle has offered RDMBS solutions, but now you can find SQL and NoSQL database solutions too.

1. Objectives:

1. To better understand the features and capabilities that make a conventional databases
2. To understand latest programming language and multiple data structures support being offered by these conventional database management systems.

4.2.2 ORACLE DATABSE MANAGEMENT SYSTEM

Oracle has provided high-quality database solutions since the 1970s. The most recent version of Oracle Database was designed to integrate with cloud-based systems, and it allows you to manage massive databases with billions of records. Moreover, Oracle lets you organize data via a "grid framework" and it uses dynamic data masking for an additional layer of security. Oracle has offered RDMBS SQL and NoSQL solutions.

4.2.3 Programming Language Support by Oracle:

Many programming languages have Object-Relational Mapping (ORM) frameworks that translate between the object-oriented data structures in programs and the relational model of databases. Such frameworks usually represent themselves as libraries to the developer that are used to abstract data interaction from the application logic. While abstracting the data access layer in this way can boost developer

productivity, the task of managing the abstraction is still the developer's responsibility. Oracle supports a large and varied development community (both internally and externally), so the company recognizes the advantages of broad language support. Today, more than 30 programming languages, including the popular languages shown in Figure 4.1, can access the various database technologies that Oracle provides. In addition, Oracle actively participates in industry-wide efforts to refine standards for database interfaces, including JDBC, Python PEP 249, and PHP, among others.

Figure 4.1: Programming languages supported by Oracle
Source: Gerald Venzl, 2017).

Oracle anticipated the popularity of REST (Representational State Transfer), a technology for creating web services, and the emerging trend of data access abstraction via REST. Oracle introduced a database component that allows RESTful access to data stored in a relational database, document store, or key-value format, making Oracle a pioneer in providing standardized REST interfaces for the data access layer. The component is known as Oracle REST Data Services (ORDS).

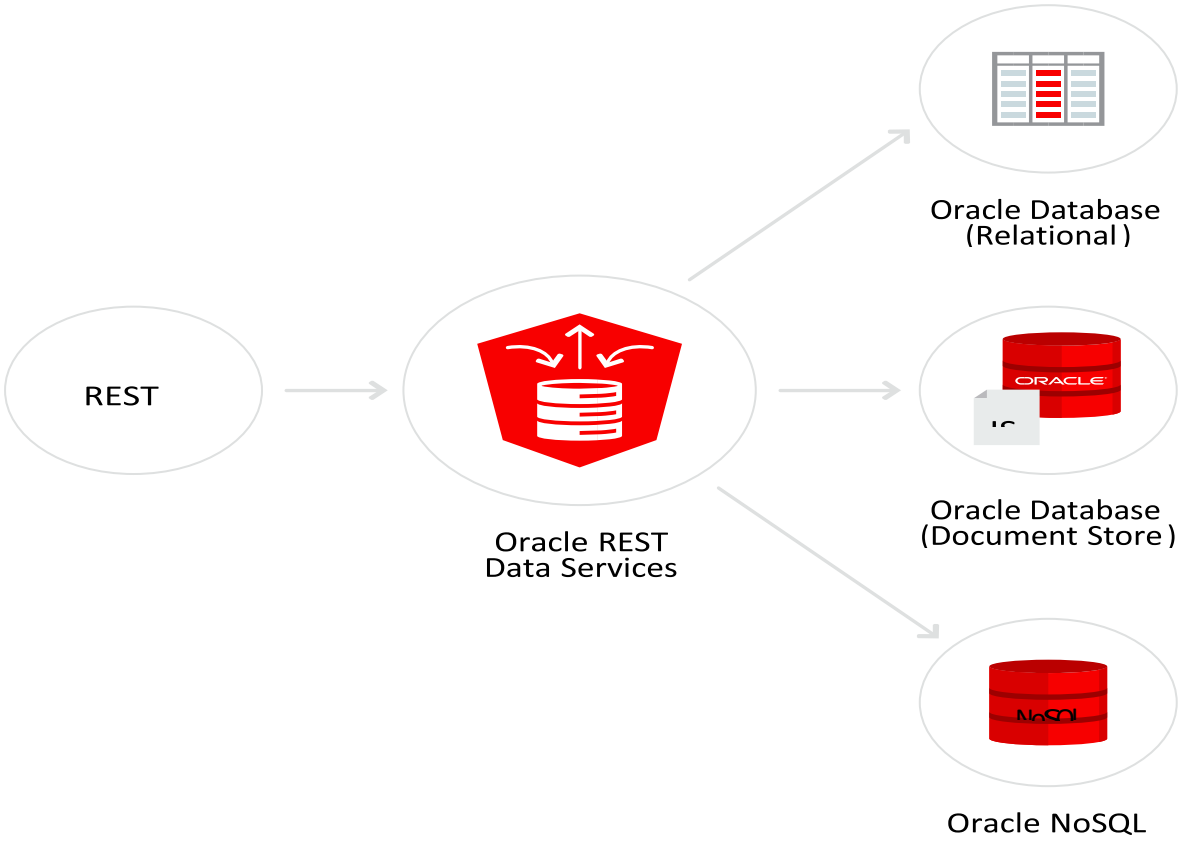


Figure 4.2*: Oracle REST Data Services*

Source: (Gerald Venzl, 2017).

4.2.4 Multi – Model Persistence:

Modern-day databases are often multi-model—that is, they give developers the choice of the desired data model without having to worry about which database to use and how to connect to it (databases that support only a single data model are known as single-model databases). Multi-model databases provide developers with a single connection method and a common API for storing and retrieving data, regardless of the data format. Data storage and retrieval is usually performed via standard SQL operations that provide the desired transparency. By using standard SQL operations, a developer can immediately leverage a multi-model database without having to refactor code or interact with a different set of APIs.

Oracle provides both single-model and multi-model database technologies. Single-model databases from Oracle include (although not exclusively) Berkeley DB, Berkeley

DB XML Data Store, Oracle NoSQL Database, and Oracle *Essbase*. Other databases from Oracle can manage multiple data models, as shown in Figure 2.3. Oracle Database 12c, for example, supports multi-model database capabilities, enabling scalable, high performance data management and analysis.

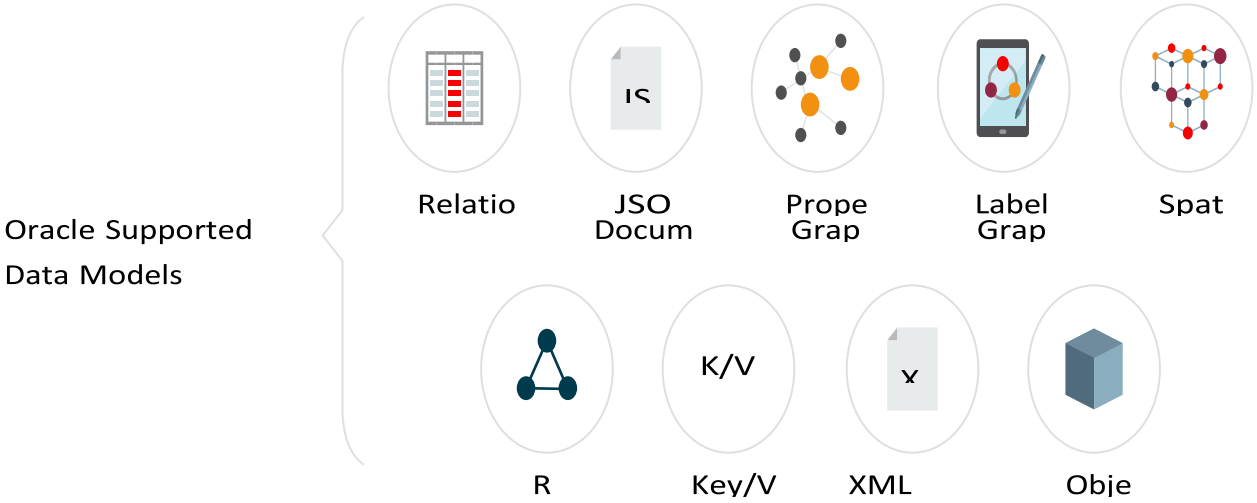


Figure 4.3: Data models supported by Oracle
Source: Gerald Venzl (2017)

4.2.5 GraalVM and the Oracle Database:

GraalVM is a universal virtual machine that runs applications written in a variety of programming languages (JavaScript, Python 3, Ruby, R, Java Virtual Machine (JVM) - based languages, and LLVM-based languages) with high performance on the same platform. Supporting multiple programming languages allows GraalVM to share basic infrastructure such as just-in-time (JIT) compilation, memory management, configuration and tooling among all supported languages. The sharing of configuration and tooling leads to a uniform developer experience in modern projects that make use of multiple programming languages.

Support for many languages is not the only benefit of GraalVM. Although it primarily runs as part of a JVM, GraalVM offers a solution for building native images written in the Java programming language. This native image feature allows to compile an entire Java application, possibly embedding multiple guest languages supported by GraalVM and including the needed virtual machine infrastructure, into a native executable or a

shared library. The main advantages of a GraalVM native image are improved startup times and reduced memory footprint. This enables existing applications and platforms not written in Java to embed GraalVM as a shared library and benefit from its universal virtual machine technology.

GraalVM provides high performance for individual languages and interoperability with zero performance overhead for creating polyglot applications. Instead of converting data structures at language boundaries, GraalVM allows objects and arrays to be used directly by foreign languages.

Example scenarios include accessing functionality of a Java library from Node.js code, calling a Python statistical routine from Java, or using R to create a complex SVG plot from data managed by another language. With GraalVM, programmers are free to use whatever language they think is most productive to solve the current task.

GraalVM 1.0 allows you to run:

1. JVM-based languages like Java, Scala, Groovy, or Kotlin
2. JavaScript (including Node.js)
3. LLVM bitcode (created from programs written in e.g. C, C++, or Rust)
4. Experimental versions of Ruby, R, and Python

GraalVM can either run standalone, embedded as part of platforms like OpenJDK or Node.js, or even embedded inside databases such as MySQL or the Oracle RDBMS. Applications can be deployed flexibly across the stack via the standardized GraalVM execution environments. In the case of data processing engines, GraalVM directly exposes the data stored in custom formats to the running program without any conversion overhead.

For JVM-based languages, GraalVM offers a mechanism to create precompiled native images with instant start up and low memory footprint. The image generation process runs a static analysis to find any code reachable from the main Java method and then performs a full ahead-of-time (AOT) compilation. The resulting native binary contains the whole program in machine code form for immediate execution. It can be

linked with other native programs and can optionally include the GraalVM compiler for complementary JIT compilation support to run any GraalVM-based language with high performance.

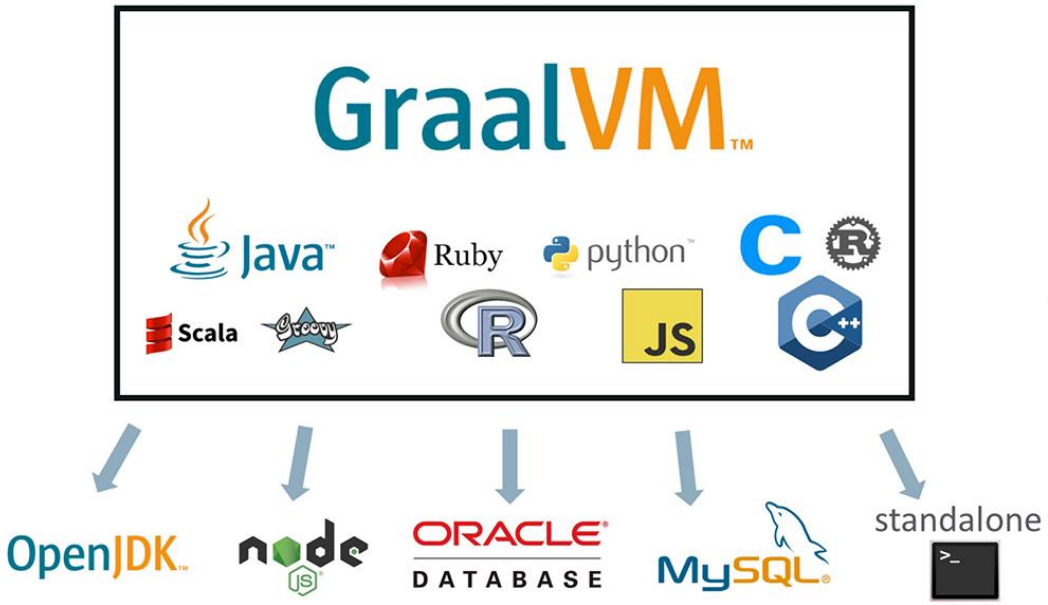


Figure 4.4: GraalVM allows multiple technologies to work with the Oracle Database

Source: https://blogs.oracle.com/developers/announcing-graalvm

A major advantage of the GraalVM ecosystem is language-agnostic tooling that is applicable in all GraalVM deployments. The core GraalVM installation provides a language-agnostic debugger, profiler, and heap viewer. We invite third-party tool developers and language developers to enrich the GraalVM ecosystem using the instrumentation API or the language-implementation API. We envision GraalVM as a language-level virtualization layer that allows leveraging tools and embeddings across all languages.

5.1 MySQL

MySQL is a free, open-source RDBMS solution that Oracle owns and manages. Even though it's freeware, MySQL benefits from frequent security and features updates. Commercial and enterprise can upgrade to paid versions of MySQL to benefit from additional features and user support. Although MySQL didn't support NoSQL in the

past, since Version 8, it provides NoSQL support to compete with other solutions like PostgreSQL.

1. Features of MySQL Database:

The following are the most important properties of MySQL.

1. **Relational Database System:** Like almost all other database systems on the market, MySQL is a relational database system.
2. **Client/Server Architecture:** MySQL is a client/server system. There is a database server (MySQL) and arbitrarily many clients (application programs), which communicate with the server; that is, they query data, save changes, etc. The clients can run on the same computer as the server or on another computer (communication via a local network or the Internet).
3. **SQL compatibility:** MySQL supports as its database language -- as its name suggests – SQL (Structured Query Language). SQL is a standardized language for querying and updating data and for the administration of a database. There are several SQL dialects (about as many as there are database systems). MySQL adheres to the current SQL standard although with significant restrictions and a large number of extensions.
4. **SubSELECTs:** Since version 4.1, MySQL is capable of processing a query in the form SELECT * FROM table1 WHERE x IN (SELECT y FROM table2) (There are also numerous syntax variants for subSELECTs.)
5. **Views:** Put simply, views relate to an SQL query that is viewed as a distinct database object and makes possible a particular view of the database. MySQL has supported views since version 5.0.
6. **Stored procedures** (SPs for short) are generally used to simplify certain steps, such as inserting or deleting a data record. For client programmers this has the advantage that they do not have to process the tables directly, but can rely on SPs. Like views, SPs help in the administration of large database projects. SPs can also increase efficiency. MySQL has supported SPs since version 5.0.

7. **Triggers:** Triggers are SQL commands that are automatically executed by the server in certain database operations (INSERT, UPDATE, and DELETE). MySQL has supported triggers in a limited form from version 5.0, and additional functionality is promised for version 5.1.
8. **Unicode:** MySQL has supported all conceivable character sets since version 4.1, including Latin-1, Latin-2, and Unicode (either in the variant UTF8 or UCS2).
9. **Full-text search:** Full-text search simplifies and accelerates the search for words that are located within a text field. If you employ MySQL for storing text (such as in an Internet discussion group), you can use full-text search to implement simply an efficient search function.
10. **Replication:** Replication allows the contents of a database to be copied (replicated) onto a number of computers. In practice, this is done for two reasons: to increase protection against system failure (so that if one computer goes down, another can be put into service) and to improve the speed of database queries.
11. **Transactions:** In the context of a database system, a transaction means the execution of several database operations as a block. The database system ensures that either all of the operations are correctly executed or none of them. This holds even if in the middle of a transaction there is a power failure, the computer crashes, or some other disaster occurs. Thus, for example, it cannot occur that a sum of money is withdrawn from account A but fails to be deposited in account B due to some type of system error.
12. **Transactions** also give programmers the possibility of interrupting a series of already executed commands (a sort of revocation). In many situations this leads to a considerable simplification of the programming process.
13. **GIS functions:** Since version 4.1, MySQL has supported the storing and processing of two-dimensional geographical data. Thus MySQL is well suited for GIS (geographic information systems) applications.
14. **Programming languages:** There are quite a number of APIs (application programming interfaces) and libraries for the development of MySQL applications.

For client programming you can use, among others, the languages C, C++, Java, Perl, PHP, and Python

15. **ODBC:** MySQL supports the ODBC interface Connector/ODBC. This allows MySQL to be addressed by all the usual programming languages that run under Microsoft Windows (Delphi, Visual Basic, etc.). The ODBC interface can also be implemented under Unix, though that is seldom necessary.
16. Windows programmers who have migrated to Microsoft's new .NET platform can, if they wish, use the ODBC provider or the .NET interface Connector/NET.
17. **Platform independence:** It is not only client applications that run under a variety of operating systems; MySQL itself (that is, the server) can be executed under a number of operating systems. The most important are Apple Macintosh OS X, Linux, Microsoft Windows, and the countless Unix variants, such as AIX, BSDI, FreeBSD, HP-UX, OpenBSD, Net BSD, SGI Iris, and Sun Solaris.
18. **Speed:** MySQL is considered a very fast database program. This speed has been backed up by a large number of benchmark tests (though such tests -- regardless of the source -- should be considered with a good dose of skepticism).

5.3 MySQL User interfaces

A graphical user interface (GUI) is a type of interface that allows users to interact with electronic devices or programs through graphical icons and visual indicators such as secondary notation, as opposed to text-based interfaces, typed command labels or text navigation. GUIs are easier to learn than command-line interfaces (CLIs), which require commands to be typed on the keyboard. Third-party proprietary and free graphical administration applications (or "front ends") are available that integrate with MySQL and enable users to work with database structure and data visually. Some well-known front ends are:

1. **MySQL Workbench:** The official integrated environment for MySQL. It was developed by MySQL AB, and enables users to graphically administer MySQL databases and visually design database structures. MySQL Workbench replaces the previous package of software, MySQL GUI Tools. Similar to other third-party

packages, but still considered the authoritative MySQL front end, MySQL Workbench lets users manage database design & modeling, SQL development (replacing MySQL Query Browser) and Database administration (replacing MySQL Administrator).

2. **MySQL Workbench** is available in two editions, the regular free and open source Community Edition which may be downloaded from the MySQL website, and the proprietary Standard Edition which extends and improves the feature set of the Community Edition.

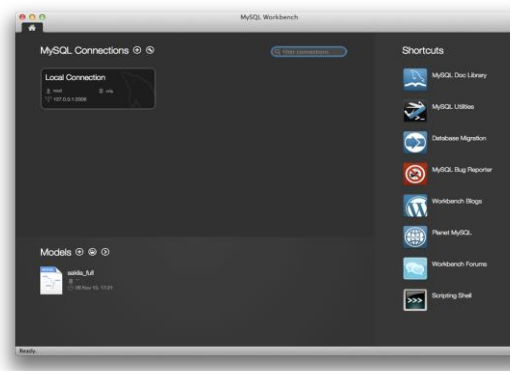


Figure 4.5: MySQL Workbench

Source: Philip Olson - Home computer, MySQL Workbench 5.2.44, GPL, https://commons.wikimedia.org/w/index.php?curid=52195090

Adminer: A free MySQL front end for managing content in MySQL databases (since version 2, it also works on PostgreSQL, Microsoft SQL Server, SQLite and Oracle databases). Adminer is distributed under the Apache License (or GPL v2) in the form of a single PHP file (around 300 KiB in size), and is capable of managing multiple databases, with many CSS skins available. Its author is Jakub Vrána who started to develop this tool as a light-weight alternative to phpMyAdmin.

3. **DBeaver:** An SQL client and a database administration tool. DBeaver includes extended support of following databases: MySQL and MariaDB, PostgreSQL,

Oracle, DB2 (LUW), Exasol, SQL Server, Sybase, Firebird, Teradata, Vertica, Apache Phoenix, Netezza, Informix, Apache Derby, H2, SQLite and any other database which has a JDBC or ODBC driver. DBeaver is free and open source software that is distributed under the Apache License 2.0. The source code is hosted on GitHub.

4. **DBEdit:** A database editor, which can connect to an Oracle, DB2, MySQL and any database that provides a JDBC driver. It runs on Windows, Linux and Solaris. DBEdit is free and open source software and distributed under the GNU General Public License. The source code is hosted on SourceForge.
5. **Navicat:** A series of graphical database management and development software produced by PremiumSoft CyberTech Ltd. for MySQL, MariaDB, Oracle, SQLite, PostgreSQL and Microsoft SQL Server. It has an Explorer-like graphical user interface and supports multiple database connections for local and remote databases. Its design is made to meet the needs of a variety of audiences, from database administrators and programmers to various businesses/companies that serve clients and share information with partners. Navicat is a cross-platform tool and works on Microsoft Windows, OS X and Linux platforms. Upon purchase, users are able to select a language for the software from eight available languages: English, French, German, Spanish, Japanese, Polish, Simplified Chinese and Traditional Chinese.
6. **phpMyAdmin:** A free and open source tool written in PHP intended to handle the administration of MySQL with the use of a web browser. It can perform various tasks such as creating, modifying or deleting databases, tables, fields or rows; executing SQL statements; or managing users and permissions. The software, which is available in 78 languages, is maintained by The phpMyAdmin Project.
7. **Toad for MySQL:** A software application from Dell Software that database developers, database administrators and data analysts use to manage both relational and non-relational databases using SQL. Toad supports many databases and environments. It runs on all 32-bit/64-bit Windows platforms, including

Microsoft Windows Server, Windows XP, Windows Vista, Windows 7 and 8 (32-Bit or 64-Bit). Dell Software has also released a Toad Mac Edition. Dell provides Toad in commercial and trial/freeware versions. The freeware version is available from the ToadWorld.com community.

5.3.1 Command-line interfaces

A command-line interface is a means of interacting with a computer program where the user issues commands to the program by typing in successive lines of text (command lines). MySQL ships with many command line tools, from which the main interface is the mysql client. Some of the command-line utilities available for MySQL are listed below:

1. **MySQL Utilities** is a set of utilities designed to perform common maintenance and administrative tasks. Originally included as part of the MySQL Workbench, the utilities are a stand-alone download available from Oracle.
2. **Percona** Toolkit is a cross-platform toolkit for MySQL, developed in Perl. **Percona** Toolkit can be used to prove replication is working correctly, fix corrupted data, automate repetitive tasks, and speed up servers. Percona Toolkit is included with several Linux distributions such as CentOS and Debian, and packages are available for Fedora and Ubuntu as well. Percona Toolkit was originally developed as Maatkit, but as of late 2011, Maatkit is no longer developed.
3. **MySQL shell** is a tool for interactive use and administration of the MySQL database. It supports JavaScript, Python or SQL modes and it can be used for administration and access purposes.

5.3.2 Advantages of using MySQL:

1. **It's free:** As an open-source RDBMS solution, MySQL is free to use in any way you want.
2. **Excellent for any size organization:** MySQL is an excellent solution for enterprise-level businesses and small startup companies alike.
3. Different user interfaces available

4. Highly compatible with other systems: MySQL has a reputation for being compatible with many other database systems.

5.3.3 Disadvantages of using MySQL:

5. Missing features common in other RDBMSs: Because MySQL prioritizes speed and agility over features, you might find that it's missing some of the standard features found in other solutions, like the ability to create incremental backups, for example.
6. Challenges getting quality support: Being a free solution, there is no dedicated support team to provide solutions for problems and challenges unless it is upgraded to the paid MySQL package from Oracle. However, MySQL does have an active volunteer community, useful forums, and lot of documentation provides answers to questions.

6.0 Microsoft SQL

Microsoft SQL Server is a relational database management system (RDBMS) that supports a wide variety of transaction processing, business intelligence and analytics applications in corporate IT environments. Like other RDBMS software, Microsoft SQL Server is built on top of SQL, a standardized programming language that database administrators (DBAs) and other IT professionals use to manage databases and query the data they contain. SQL Server is tied to Transact-SQL (T-SQL), an implementation of SQL from Microsoft that adds a set of proprietary programming extensions to the standard language. The core component of Microsoft SQL Server is the SQL Server Database Engine, which controls data storage, processing and security. It includes a relational engine that processes commands and queries and a storage engine that manages database files, tables, pages, indexes, data buffers and transactions. Stored procedures, triggers, views and other database objects are also created and executed by the Database Engine.

6.1 Microsoft SQL Services

SQL Server also includes an assortment of add-on services. While these are not essential for the operation of the database system, they provide value added services on top of the core database management system. These services either run as a part of some SQL Server component or out-of-process as Windows Service and presents their own API to control and interact with them.

6.1.1 Machine Learning Services:

The SQL Server Machine Learning services operates within the SQL server instance, allowing people to do machine learning and data analytics without having to send data across the network or be limited by the memory of their own computers. The services come with Microsoft's R and Python distributions that contain commonly used packages for data science, along with some proprietary packages (e.g. revoscalepy, RevoScaleR, microsoftml) that can be used to create machine models at scale. Analysts can either configure their client machine to connect to a remote SQL server and push the script executions to it, or they can run a R or Python scripts as an external script inside a T-SQL query. The trained machine learning model can be stored inside a database and used for scoring.

6.1.2 Service Broker: Used inside an instance, programming environment. For cross-instance applications, Service Broker communicates over TCP/IP and allows the different components to be synchronized, via exchange of messages. The Service Broker, which runs as a part of the database engine, provides a reliable messaging and message queuing platform for SQL Server applications. Service broker services consists of the following parts:

1. message types
2. contracts
3. queues
4. service programs

5. routes

The message type defines the data format used for the message. This can be an XML object, plain text or binary data, as well as a null message body for notifications. The contract defines which messages are used in a conversation between services and who can put messages in the queue. The queue acts as storage provider for the messages. They are internally implemented as tables by SQL Server, but don't support insert, update, or delete functionality. The service program receives and processes service broker messages. Usually the service program is implemented as stored procedure or CLR application. Routes are network addresses where the service broker is located on the network. Also, service broker supports security features like network authentication (using NTLM, Kerberos, or authorization certificates), integrity checking, and message encryption.

6.1.3 Reporting Services:

SQL Server Reporting Services is a report generation environment for data gathered from SQL Server databases. It is administered via a web interface. Reporting services features a web services interface to support the development of custom reporting applications. Reports are created as RDL files. Reports can be designed using recent versions of Microsoft Visual Studio (Visual Studio.NET 2003, 2005, and 2008) with Business Intelligence Development Studio, installed or with the included Report Builder. Once created, RDL files can be rendered in a variety of formats, including Excel, PDF, CSV, XML, BMP, EMF, GIF, JPEG, PNG, and TIFF and HTML Web Archive.

6.3.4 Full Text Search Service:

SQL Server Full Text Search service is a specialized indexing and querying service for unstructured text stored in SQL Server databases. The full text search index can be created on any column with character-based text data. It allows for words to be searched for in the text columns. While it can be performed with the SQL LIKE operator, using SQL Server Full Text Search service can be more efficient. It allows for

inexact matching of the source string, indicated by a Rank value which can range from 0 to 1000—a higher rank means a more accurate match. It also allows linguistic matching ("inflectional search"), i.e., linguistic variants of a word (such as a verb in a different tense) will also be a match for a given word (but with a lower rank than an exact match). Proximity searches are also supported, i.e., if the words searched for do not occur in the sequence they are specified in the query but are near each other, they are also considered a match. T-SQL exposes special operators that can be used to access the FTS capabilities.

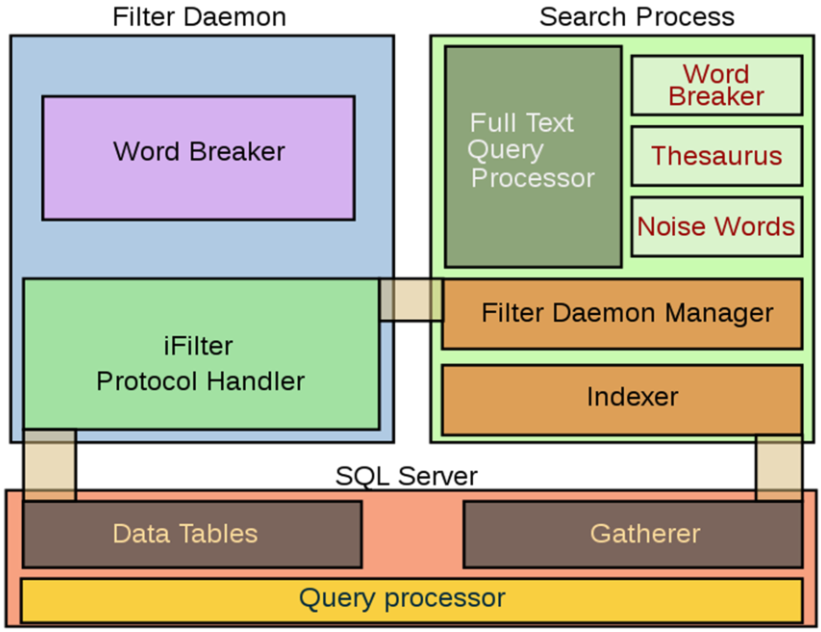


Figure 4.6: SQL Server Full Text Search Architecture

The Full Text Search engine is divided into two processes: The Filter Daemon process (msftefd.exe) and the Search process (msftesql.exe). These processes interact with the SQL Server. The Search process includes the indexer (that creates the full text indexes) and the full text query processor. The indexer scans through text columns in the database. It can also index through binary columns, and use iFilters to extract meaningful text from the binary blob (for example, when a Microsoft Word document is stored as an unstructured binary file in a database). The iFilters are hosted by the Filter Daemon process. Once the text is extracted, the Filter Daemon process breaks it up into a sequence of words and hands it over to the indexer. The indexer filters out

noise words, i.e., words like A, And etc., which occur frequently and are not useful for search. With the remaining words, an inverted index is created, associating each word with the columns they were found in. SQL Server itself includes a Gatherer component that monitors changes to tables and invokes the indexer in case of updates. When a full text query is received by the SQL Server query processor, it is handed over to the FTS query processor in the Search process. The FTS query processor breaks up the query into the constituent words, filters out the noise words, and uses an inbuilt thesaurus to find out the linguistic variants for each word. The words are then queried against the inverted index and a rank of their accurateness is computed. The results are returned to the client via the SQL Server process.

6.1.5 Notification Services:

Originally introduced as a post-release add-on for SQL Server 2000, Notification Services was bundled as part of the Microsoft SQL Server platform for the first and only time with SQL Server 2005. SQL Server Notification Services is a mechanism for generating data-driven notifications, which are sent to Notification Services subscribers. A subscriber registers for a specific event or transaction (which is registered on the database server as a trigger); when the event occurs, Notification Services can use one of three methods to send a message to the subscriber informing about the occurrence of the event.

6.4 Important SQL Server Tools:

6.4.1 SQLCMD:

A command line application that comes with Microsoft SQL Server, and exposes the management features of SQL Server. It allows SQL queries to be written and executed from the command prompt. It can also act as a scripting language to create and run a set of SQL statements as a script. Such scripts are stored as a SQL file, and are used either for management of databases or to create the database schema during the deployment of a database. SQLCMD was introduced with SQL Server 2005 and has continued through SQL Server versions 2008, 2008 R2, 2012, 2014, 2016 and 2019. Its

predecessor for earlier versions was OSQL and ISQL, which were functionally equivalent as it pertains to TSQL execution, and many of the command line parameters are identical, although SQLCMD adds extra versatility.

6.4.2 **Visual Studio:**

A programming tool that includes native support for data programming with Microsoft SQL Server. It can be used to write and debug code to be executed by SQL CLR. It also includes a data designer that can be used to graphically create, view or edit database schemas. Queries can be created either visually or using code. SSMS 2008 onwards, provides intelligence for SQL queries as well.

6.4.3 SQL Server Management Studio:

A Graphic User Interface (GUI) tool included with SQL Server 2005 and later for configuring, managing, and administering all components within Microsoft SQL Server. The tool includes both script editors and graphical tools that work with objects and features of the server. SQL Server Management Studio replaces Enterprise Manager as the primary management interface for Microsoft SQL Server since SQL Server 2005. A version of SQL Server Management Studio is also available for SQL Server Express Edition, for which it is known as SQL Server Management Studio Express (SSMSE).

A central feature of SQL Server Management Studio is the Object Explorer, which allows the user to browse, select, and act upon any of the objects within the server. It can be used to visually observe and analyze query plans and optimize the database performance, among others. SQL Server Management Studio can also be used to create a new database, alter any existing database schema by adding or modifying tables and indexes, or analyze performance. It includes the query windows which provide a GUI based interface to write and execute queries.

6.4.4 SQL Server Operations Studio:

A cross platform query editor available as an optional download. The tool allows users to write queries; export query results; commit SQL scripts to GIT repositories and perform basic server diagnostics. SQL Server Operations Studio supports Windows, Mac and Linux systems. It was released to General Availability in September 2018, at which point it was also renamed to Azure Data Studio. The functionality remains the same as before.

6.4.5 Business Intelligence Development Studio (BIDS): The Integrated Development Environment (IDE) from Microsoft used for developing data analysis and Business Intelligence solutions utilizing the Microsoft SQL Server Analysis Services, Reporting Services and Integration Services. It is based on the Microsoft Visual Studio development environment but is customized with the SQL Server services-specific extensions and project types, including tools, controls and projects for reports (using Reporting Services), Cubes and data mining structures (using Analysis Services. For SQL Server 2012 and later, this IDE has been renamed SQL Server Data Tools (SSDT).

6.5 Advantages of using Microsoft SQL Server include:

1. **Mobilility:** This database engine allows you to access dashboard graphics and visuals via mobile devices.
2. **Integrates with Microsoft products:** Companies that rely heavily on Microsoft products will enjoy the way SQL Server integrates easily with these applications.
3. **Speed:** Microsoft SQL Server has built a reputation around being fast and stable.

6.6 Disadvantages of using Microsoft SQL Server:

1. **Expensive:** Considering that there are plenty of free database engines available, the cost of Microsoft SQL Server is steep. It's over N14,000 for one enterprise-level license per core. There are scaled down licensing options for approximately N3,700 and N900, and a free version you can use to test the platform.

2. **Requires a lot of resources**: This resource-heavy RDBMS may require you to purchase better hardware.

7.1 MongoDB

MongoDB is a free, open-source database engine built especially for applications that use unstructured data. Because most DBMSs were built for structured data—even if add-ons allow them to handle non-relational data now—MongoDB often excels where other DBMSs fail. MongoDB works with structured data too, but since this database engine wasn't designed for relational data, performance slowdowns are likely.

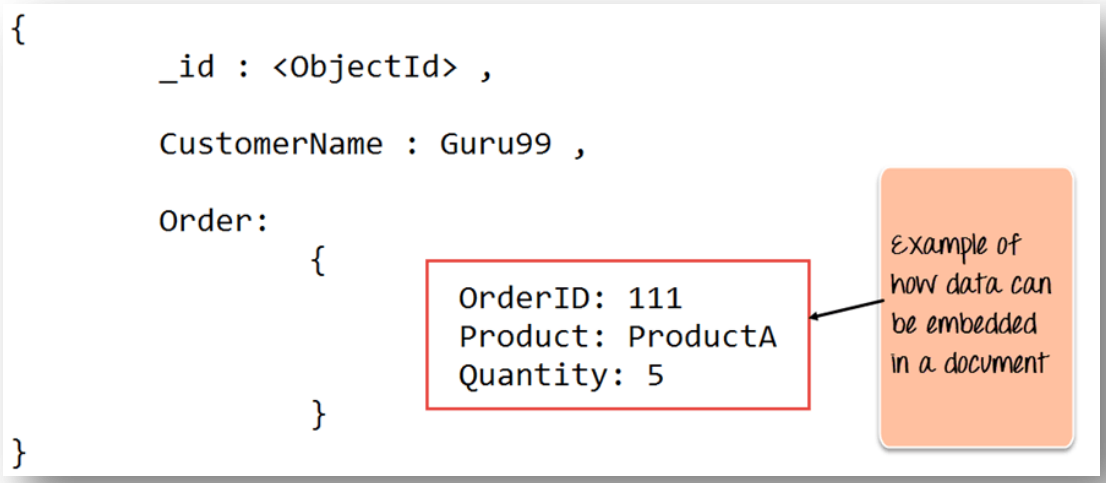
7.2 MongoDB Features:

1. Each database contains collections which in turn contains documents. Each document can be different with a varying number of fields. The size and content of each document can be different from each other.
2. The document structure is more in line with how developers construct their classes and objects in their respective programming languages. Developers will often say that their classes are not rows and columns but have a clear structure with key-value pairs.
3. As seen in the introduction with NoSQL databases, the rows (or documents as called in MongoDB) doesn't need to have a schema defined beforehand. Instead, the fields can be created on the fly.
4. The data model available within MongoDB allows you to represent hierarchical relationships, to store arrays, and other more complex structures more easily.
5. **Scalability** – The MongoDB environments are very scalable. Companies across the world have defined clusters with some of them running 100+ nodes with around millions of documents within the database.

7.3 MongoDB Example:

The below example shows how a document can be modeled in MongoDB.

1. The _id field is added by MongoDB to uniquely identify the document in the collection.
2. What you can note is that the Order Data (OrderID, Product, and Quantity) which in RDBMS will normally be stored in a separate table, while in MongoDB it is actually stored as an embedded document in the collection itself. This is one



of the key differences in how data is modelled in MongoDB.

Figure 4.7: MongoDB example
Source: https://www.guru99.com/what-is-mongodb.html

7.4 Key Components of MongoDB Architecture:

The components of MongoDB

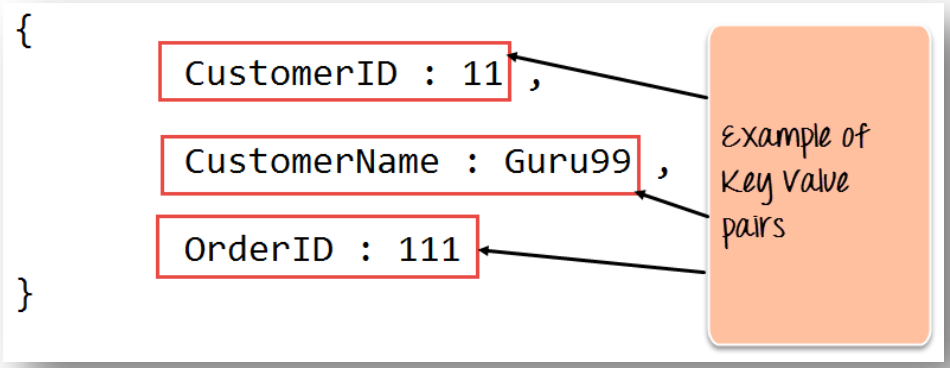
1. **_id** – This is a field required in every MongoDB document. The _id field represents a unique value in the MongoDB document. The _id field is like the document's primary key. If you create a new document without an _id field, MongoDB will automatically create the field. So for example, if we see the example of the above customer table, Mongo DB will add a 24 digit unique identifier to each document in the collection.

Table 5.1: Key features of MongoDB architecture

_Id	CustomerID	CustomerName	OrderID
563479cc8a8a4246bd27d784	11	Guru99	111
563479cc7a8a4246bd47d784	22	Trevor Smith	222
563479cc9a8a4246bd57d784	33	Nicole	333

1. **Collection** – This is a grouping of MongoDB documents. A collection is the equivalent of a table which is created in any other RDMS such as Oracle or MS SQL. A collection exists within a single database. As seen from the introduction collections don't enforce any sort of structure.
2. **Cursor** – This is a pointer to the result set of a query. Clients can iterate through a cursor to retrieve results.
3. **Database** – This is a container for collections like in RDMS wherein it is a container for tables. Each database gets its own set of files on the file system. A MongoDB server can store multiple databases.
4. **Document** - A record in a MongoDB collection is basically called a document. The document, in turn, will consist of field name and values.
5. **Field** - A name-value pair in a document. A document has zero or more fields. Fields are analogous to columns in relational databases.

The following diagram shows an example of Fields with Key value pairs. So in the



example below CustomerID and 11 is one of the key value pair's defined in the document.

Figure 4.7: Example of fields with key-value pairs

Source: https://www.guru99.com/what-is-mongodb.html

1. **JSON** – This is known as JavaScript Object Notation. This is a human-readable, plain text format for expressing structured data. JSON is currently supported in many programming languages. Just a quick note on the key difference between the _id field and a normal collection field. The _id field is used to uniquely identify the documents in a collection and is automatically added by MongoDB when the collection is created.

7.5 Why Use MongoDB?:

The reasons as to why one should start using MongoDB

1. **Document-oriented** – Since MongoDB is a NoSQL type database, instead of having data in a relational type format, it stores the data in documents. This makes MongoDB very flexible and adaptable to real business world situation and requirements.

2. **Ad hoc queries** - MongoDB supports searching by field, range queries, and regular expression searches. Queries can be made to return specific fields within documents.
3. **Indexing** - Indexes can be created to improve the performance of searches within MongoDB. Any field in a MongoDB document can be indexed.
4. **Replication** - MongoDB can provide high availability with replica sets. A replica set consists of two or more mongo DB instances. Each replica set member may act in the role of the primary or secondary replica at any time. The primary replica is the main server which interacts with the client and performs all the read/write operations. The Secondary replicas maintain a copy of the data of the primary using built-in replication. When a primary replica fails, the replica set automatically switches over to the secondary and then it becomes the primary server.
5. **Load balancing** - MongoDB uses the concept of sharding to scale horizontally by splitting data across multiple MongoDB instances. MongoDB can run over multiple servers, balancing the load and/or duplicating data to keep the system up and running in case of hardware failure.

7.6 Data Modelling in MongoDB:

As we have seen from the section 7.1, the data in MongoDB has a flexible schema. Unlike in SQL databases, where you must have a table's schema declared before inserting data, MongoDB's collections do not enforce document structure. This sort of flexibility is what makes MongoDB so powerful.

When modeling data in Mongo, keep the following things in mind

1. What are the needs of the application – Look at the business needs of the application and see what data and the type of data needed for the application. Based on this, ensure that the structure of the document is decided accordingly.
2. What are data retrieval patterns – If you foresee a heavy query usage then consider the use of indexes in your data model to improve the efficiency of queries.

3. Are frequent inserts, updates and removals happening in the database – Reconsider the use of indexes or incorporate sharding if required in your data modeling design to improve the efficiency of your overall MongoDB environment.

7.8 Difference between MongoDB & RDBMS

Below are some of the key term differences between MongoDB and RDBMS

Table 5.2: Differences between MongoDB and RDBMS

RDBMS	MongoDB	Difference
Table	Collection	In RDBMS, the table contains the columns and rows which are used to store the data whereas, in MongoDB, this same structure is known as a collection. The collection contains documents which in turn contains Fields, which in turn are key-value pairs.
Row	Document	In RDBMS, the row represents a single, implicitly structured data item in a table. In MongoDB, the data is stored in documents.
Column	Field	In RDBMS, the column denotes a set of data values. These in MongoDB are known as Fields.
Joins	Embedded documents	In RDBMS, data is sometimes spread across various tables and in order to show a complete view of all data, a join is sometimes formed across tables to get the data. In MongoDB, the data is normally stored in a single collection, but separated by using Embedded documents. So there is no concept of joins in MongoDB.

Apart from the terms differences, a few other differences are shown below

1. Relational databases are known for enforcing data integrity. This is not an explicit requirement in MongoDB.
2. RDBMS requires that data be normalized first so that it can prevent orphan records and duplicates Normalizing data then has the requirement of more tables, which will then result in more table joins, thus requiring more keys and indexes.

7.9 Conclusion:

MongoDB connects non-relational databases with applications by using a wide variety of drivers (based on the programming language of the application). The most recent versions of MongoDB include pluggable storage engines. Upgraded text search features are also available, along with partial indexing features which can help with performance.

7.10 Tutor Marked Assignment

1. What are the key components of MongoDB?
2. Describe the services provided by Microsoft SQL
3. State the advantages of using Microsoft SQL Server

7.11 Further Reading and References:

Larry Ellison Introduces 'A Big Deal': The Oracle Autonomous Database". Forbes. 2 October 2019. Archived from the original on 22 December 2017. Retrieved 18 December 2019

MSDN: Introducing SQL Server Management Studio". Msdn.microsoft.com. Retrieved 2011-09-04.

Banker, Kyle (March 28, 2011), MongoDB in Action (1st ed.), Manning, p. 375, ISBN 978-1-935182-87-0

Ben-Gan, Itzik, et al. (2006). Inside Microsoft SQL Server 2005: T-SQL Programming. Microsoft Press. ISBN 0-7356-2197-7.

Chodorow, Kristina; Dirolf, Michael (September 23, 2010), MongoDB: The Definitive Guide (1st ed.), O'Reilly Media, p. 216, ISBN 978-1-4493-8156-1

Delaney, Kalen, et al. (2007). Inside SQL Server 2005: Query Tuning and Optimization. Microsoft Press. ISBN 0-7356-2196-9.

Gerald Venzl (2017). Database Requirements for Modern Developers

Giles, Dominic (13 February 2019). "Oracle Database 19c : Now available on Oracle Exadata". Archived from the original on 10 May 2019. Retrieved 10 May 2019.

Hawkins, Tim; Plugge, Eelco; Membrey, Peter (September 26, 2010), The Definitive Guide to MongoDB: The NoSQL Database for Cloud and Desktop Computing (1st ed.), Apress, p. 350, ISBN 978-1-4302-3051-9

Klaus Aschenbrenner (2011). "Introducing Service Broker". Pro SQL Server 2008 Service Broker. Vienna: Apress. p. 17-31. ISBN 978-1-4302-0865-5. Retrieved

Lance Delano, Rajesh George et al. (2005). Wrox's SQL Server 2005 Express Edition Starter Kit (Programmer to Programmer). Microsoft Press. ISBN 0-7645-8923-7.

MariaDB versus MySQL - Compatibility". MariaDB KnowledgeBase. Retrieved 16 September 2016.

MongoDB World". www.mongodb.com. Archived from the original on April 26, 2019. Retrieved April 10, 2019.

MySQL Federated Tables: The Missing Manual". O'Reilly Media. 8 October 2006. Retrieved 1 February 2012.

Pirtle, Mitch (March 3, 2011), MongoDB for Web Development (1st ed.), Addison-Wesley Professional, p. 360, ISBN 978-0-321-7053

Translations". phpMyAdmin. Retrieved 23 December 2019